

62%

wins are ensemble stacking

35/45

GBM the dominant model

$\kappa = 0.65$

coding reliability

45%

winner-control author overlap

5 of 6

folklore couplings unsupported

Abstract

Does elaborate feature engineering win tabular Kaggle competitions? This study codes 45 winners and 11 near-winning controls across a 53-technique taxonomy. Three findings:

- Winning reduces to a few **paradigms**: ensemble stacking dominates (28/45).
- Winning feature engineering is a **small shared core** that does *not cleanly* separate winners from near-winners ($n = 11$, n.s.).
- The documented top is a small recurring **guild**.

Winning turns less on FE *volume* than on paradigm fit and community membership.

Introduction & Motivation

“What wins on Kaggle” is mostly **anecdotal**. Benchmarks compare *algorithms* on fixed data, never winners against strong non-winners. Both are coded here.

Research Questions

- RQ1:** What *paradigms* define winning? Do folklore constraint→strategy rules hold?
- RQ2:** Which FE techniques, and how much?
- RQ3:** Does FE *distinguish* winners from strong non-winners?
- RQ4:** How is the winning population *structured*?

Data Description

Attribute	Description
Source	Kaggle (Playground S3–S6, TPS 2022, 1 Featured)
Sample size	45 winners + 11 near-winning controls
Variables	40-column schema + 53-technique FE taxonomy
Time range	Feb 2022 – Apr 2026
Selection	voluntary writeups ⇒ documentation-self-selection bias

Methodology

Three-pass content-analysis coding of each solution, with a near-winner *control group* and a *blind reliability re-code*.

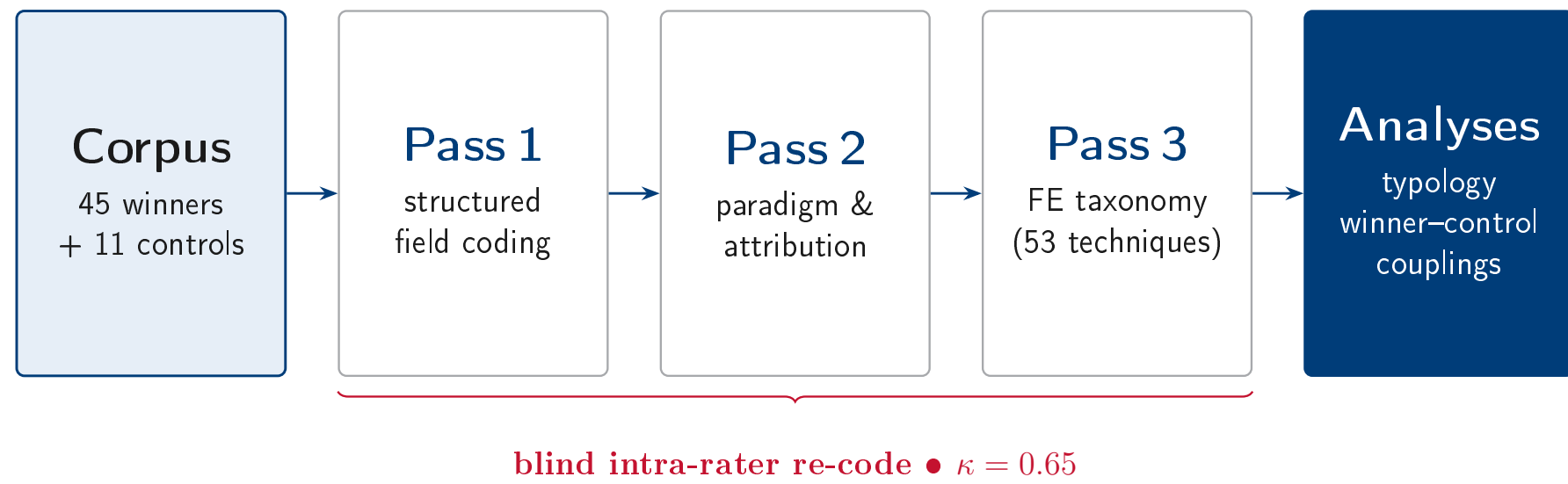


Figure 1. Multi-pass coding workflow with a near-winner control group and reliability check.

Analytical Framework

No model is trained. The unit is one completed solution (no RMSE/accuracy). The framework:

- Four paradigms:** ensemble-stacking, single-model-FE, lookup-exploit, problem-fit-NN.
- Winner-control design:** paired FE comparison (sign test).
- Coupling audit:** six folklore “constraint→strategy” claims tested.

Each winner is tested against a **near-winning control** from the *same* competition. That comparison isolates what distinguishes *winning* from merely strong play.

Implementation & Tools

Component	Tools / Libraries
Language / env	Python 3.13, Poetry
Analysis	pandas, numpy, Python standard library
Collection	Kaggle API, requests + BeautifulSoup
Coding	anthropic SDK + Claude Code (AI-assisted, researcher-directed)
Visualization	matplotlib

Inside Winning Solutions

Sized by how many of the 45 winners use each: a few models and **ensemble stacking** dominate, while feature engineering is a long, varied tail.

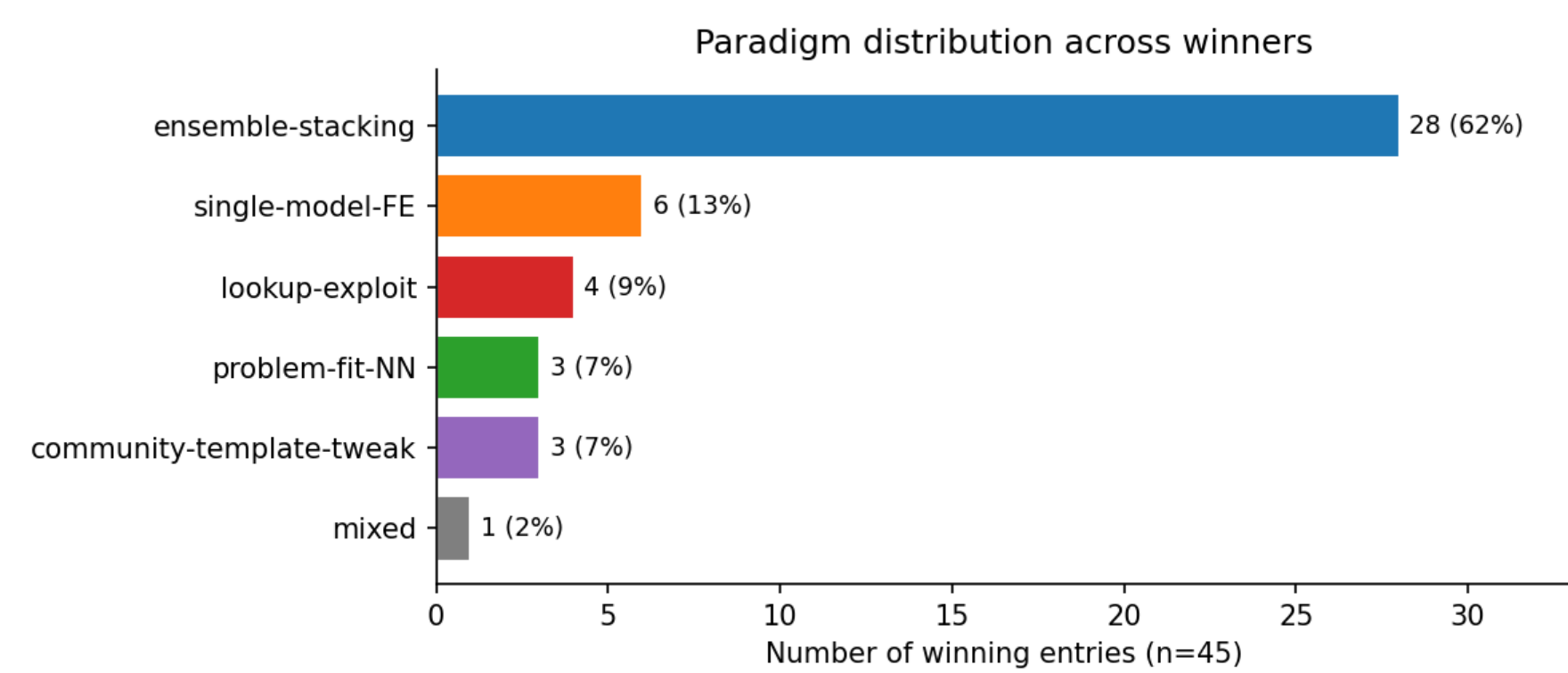


Figure 2. Technique vocabulary (word size = winners using it) and the paradigm split.

Results

FE does not cleanly separate winners from near-winners ($n = 11$, not significant); only *learned* features lean winner.

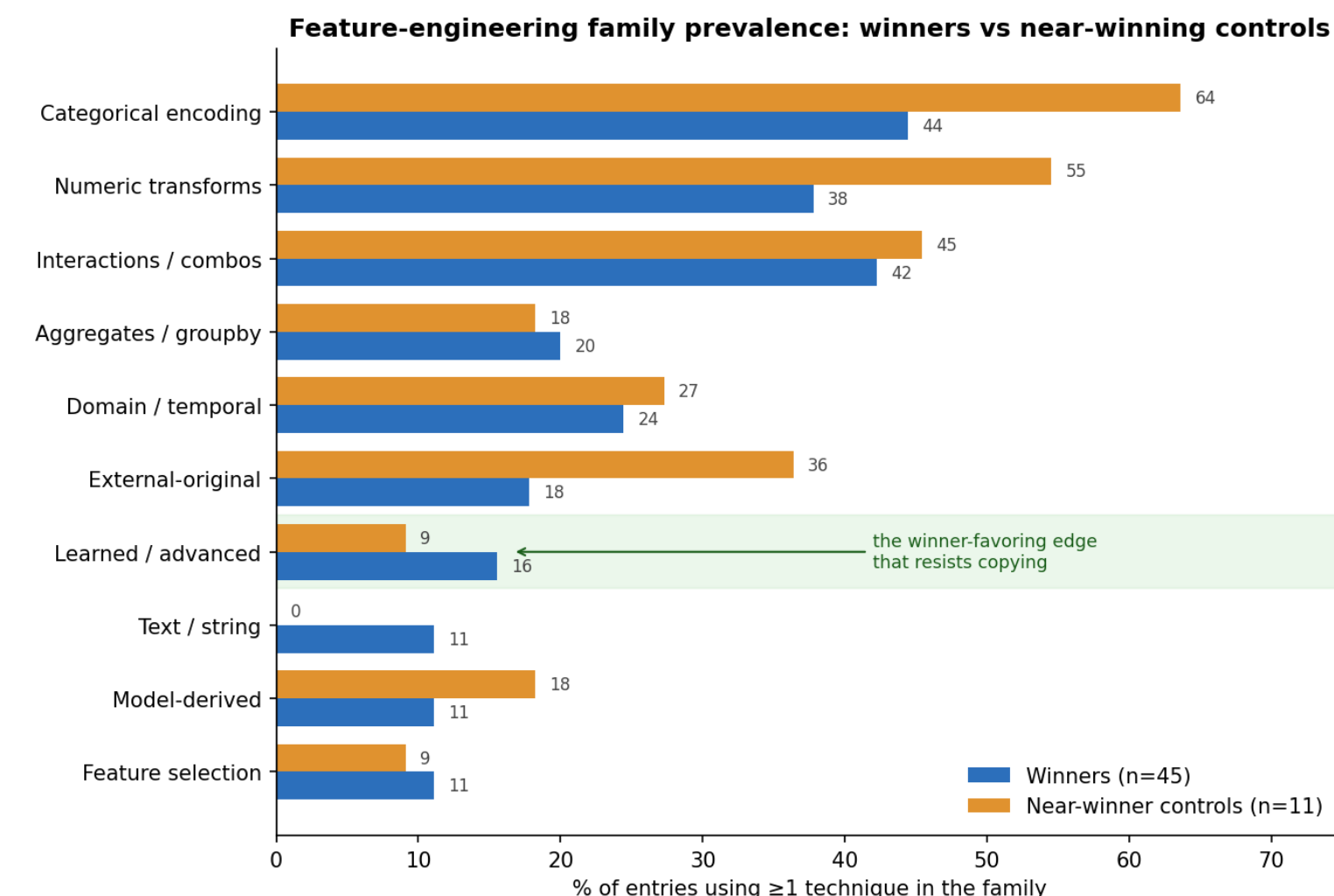


Figure 3. FE-family prevalence, winners vs. near-winning controls.

5 of 6 couplings aren’t supported. Only linear-stacker dominance holds, and it just restates standard ensembling.

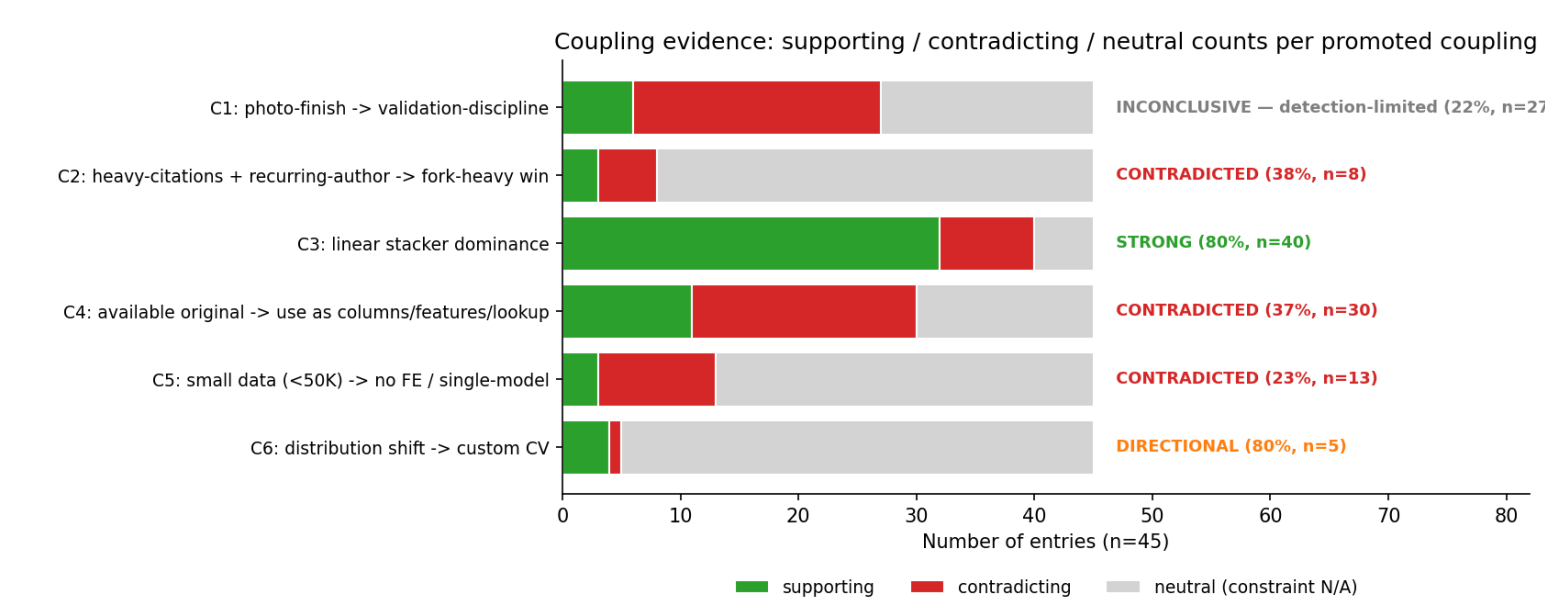


Figure 4. Coupling-evidence verdicts across the six promoted couplings.

Community Structure

A **small recurring guild**. A few authors recur across winners and near-winners; some win, others mainly supply widely-cited techniques (originators vs. canonizers).

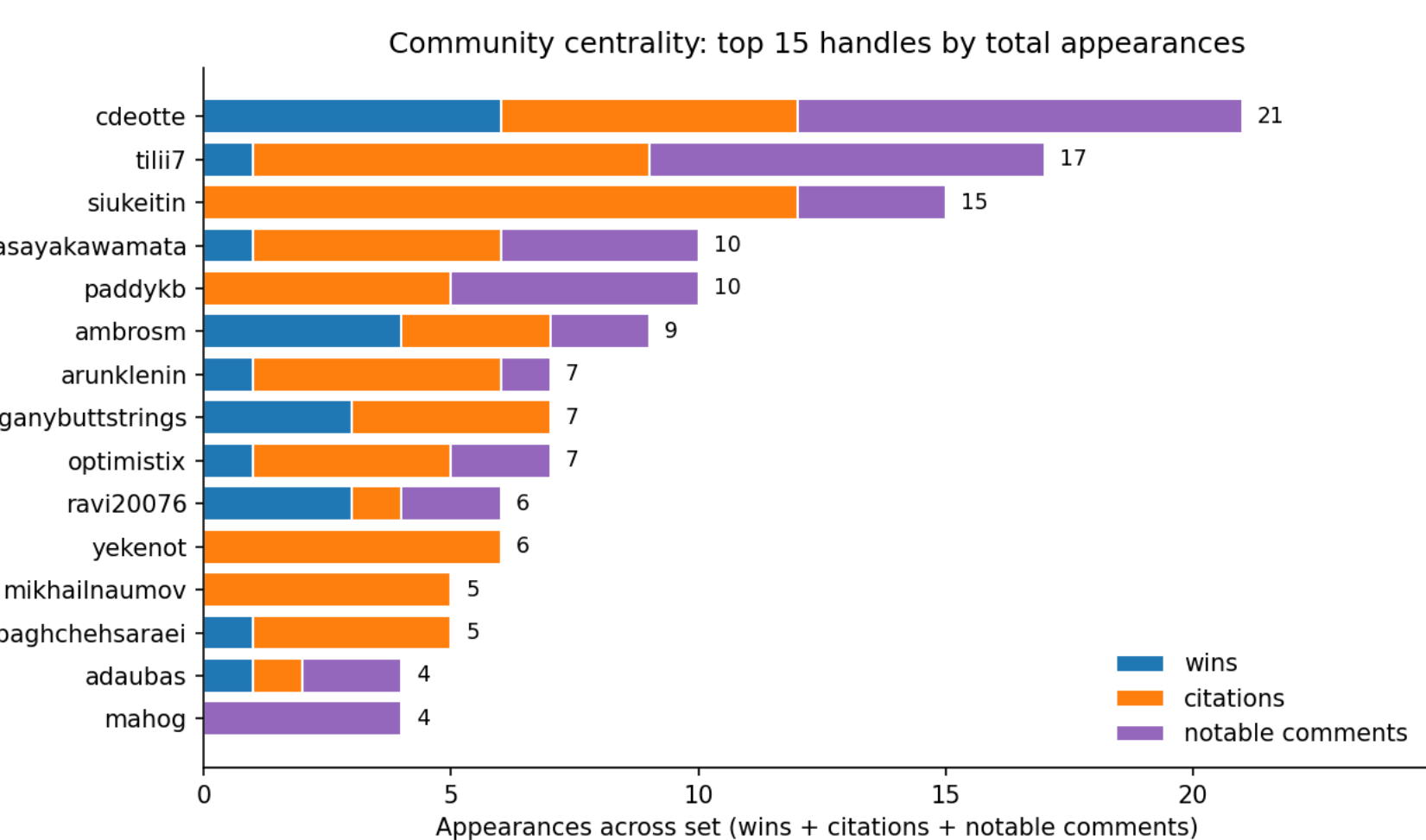


Figure 5. Most-central community members, by wins + citations + notable comments.

Discussion & Conclusions

Key Takeaways

- Winning ≠ FE folklore: a small shared core, not elaborate craft.
- Learned features:** the one edge that leans to winners.
- The top is a **recurring guild** (45% author overlap), partly about *who competes*.

Limitations: small n ; synthetic-data corpus; intra-rater reliability; documentation-self-selection.

Future Work: real-data Featured competitions; other platforms and modalities.

Acknowledgments & Contact

The author thanks **Dr. Pedro Albuquerque** for his guidance throughout CS 495, and the Bellevue College Computer Science program for supporting this capstone.

Contact: kenneth.young@bellevuecollege.edu

Code: github.com/kennethyoung/CS495-Mapping-Best-Empirical-Models